

The chaos of the modern deep learning training stack, can Nix fix it?



Gaétan Lepage

September 11, 2026

Grenug meetup #1

Who am I?

Gaétan Lepage

- Ensimag 2020
- PhD @Inria Grenoble (RobotLearn team)
 - Deep Learning for robotic acoustics
- Nix(OS) contributor since 2021
 - Python ecosystem maintenance
 - Member of the CUDA team
- Research Platform Engineer @Mistral.ai



NixOS

ExploreDownloadLearnGovernance**Community**BlogDonate

← Back

CUDA Team

The CUDA team is responsible for everything related to [CUDA](#) in [Nixpkgs](#).

Motivation

GPU-acceleration has become a fundamental prerequisite to machine learning work and scientific computing. Without support for GPU-acceleration, the Nixpkgs ecosystem will remain largely invisible to practitioners.

Nixpkgs and NixOS give us the tools we need to build reproducible and robust packages and environments. However, the state of scientific computing in Nixpkgs does not reflect this.

Goals and Responsibilities

The singular focus of the CUDA-maintainers team is to make Nixpkgs the best repository of packages leveraging CUDA, especially in machine learning and scientific computing. Towards that end, we seek to:

- improve the quantity and quality of CUDA-enabled packages in Nixpkgs
- document CUDA packaging best practices, guidelines, and patterns
- support community efforts involving packaging or distributing CUDA-enabled software

Members

Serge K ([@sergek](#))

Connor Baker ([@connorbaker](#))

Gaétan Lepage ([@glepage](#))

Yuriy Taraday ([@YorikSar](#))

Contact

- [Discourse](#)
- [Matrix](#)
- [GitHub project board](#)

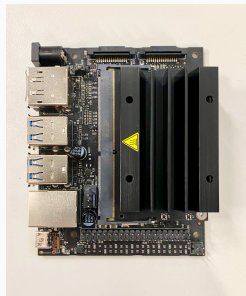
Overview of the modern Deep Learning software stack

Overview of the modern Deep Learning software stack

- Hardware
- More Hardware
- Software layers
- Packaging



Jetson AGX Orin



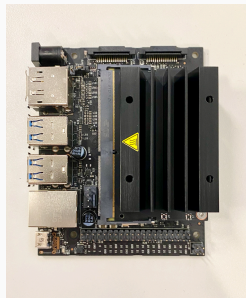
Jetson Nano

Two major types of deployment:

- Embedded systems (Jetson)



Jetson AGX Orin



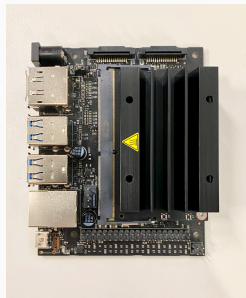
Jetson Nano

Two major types of deployment:

- Embedded systems (Jetson)
- Data centers (training, inference)



Jetson AGX Orin



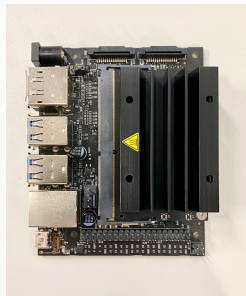
Jetson Nano

Two major types of deployment:

- Embedded systems (Jetson)
- Data centers (training, inference)
- (well your gaming PC too)



Jetson AGX Orin



Jetson Nano

Two major types of deployment:

- Embedded systems (Jetson)
- Data centers (training, inference)
- (well your gaming PC too)

We will focus on NVIDIA, because the rest is well... not as popular (love you too AMD)

...to the datacenter



DC:

- CPUs (x86_64, aarch64) (NVIDIA Grace & Vera)



DC:

- CPUs (x86_64, aarch64) (NVIDIA Grace & Vera)
- GPUs (different architectures (Hopper, Blackwell, Rubin, ...))



DC:

- CPUs (**x86_64**, **aarch64**) (NVIDIA Grace & Vera)
- GPUs (different architectures (Hopper, Blackwell, Rubin, ...))
- Well, multiple GPUs per node (intra-node communication)



DC:

- CPUs (**x86_64**, **aarch64**) (NVIDIA Grace & Vera)
- GPUs (different architectures (Hopper, Blackwell, Rubin, ...))
- Well, multiple GPUs per node (intra-node communication)
- Well, of course many nodes (cooling, networking, switches, inter-node communication, ...)



DC:

- CPUs (**x86_64**, **aarch64**) (NVIDIA Grace & Vera)
- GPUs (different architectures (Hopper, Blackwell, Rubin, ...))
- Well, multiple GPUs per node (intra-node communication)
- Well, of course many nodes (cooling, networking, switches, inter-node communication, ...)
- Storage (a lot)

Getting to software... Infrastructure at scale.

The DC is not your homelab

Getting to software... Infrastructure at scale.

The DC is not your homelab

- Orchestration (Kubernetes, ...)

Getting to software... Infrastructure at scale.

The DC is not your homelab

- Orchestration (Kubernetes, ...)
- Scheduling (SLURM, ...)

Getting to software... Infrastructure at scale.

The DC is not your homelab

- Orchestration (Kubernetes, ...)
- Scheduling (SLURM, ...)
- Firmware

Getting to software... Infrastructure at scale.

The DC is not your homelab

- Orchestration (Kubernetes, ...)
- Scheduling (SLURM, ...)
- Firmware
- Drivers (expose the GPU capabilities to software)
 - Kernel module (closed source, then open source)
 - User-space ([libcuda.so](#))

Large scale deep learning stacks are very complex.

Several languages at play:

Large scale deep learning stacks are very complex.

Several languages at play:

- CUDA, PTXAS

Large scale deep learning stacks are very complex.

Several languages at play:

- CUDA, PTXAS
- C/C++ (low-level system for interfacing with CUDA and being performant)

Large scale deep learning stacks are very complex.

Several languages at play:

- CUDA, PTXAS
- C/C++ (low-level system for interfacing with CUDA and being performant)
- Python

-> Many levels of abstractions

-> Performance is absolutely critical

Dependency hell

Overview of the modern stack:

- CUDA toolkit (libraries: cuBLAS, cuDNN, cuSPARSE, ...)
- Communication libraries (RDMA, NCCL, MPI, ...)
- Other system libraries (ffmpeg, Magma, ...)
- Kernel libraries (Flash Attention, FlashMLA, ThunderKittens, ...)
- Kernel frameworks (Triton, cuTile, CUTLASS DSL, ...)
- Deep Learning framework (PyTorch, JAX, ...)
- Domain specific libraries (torchaudio, ...)
- Sandboxing frameworks (microvms, ray, ...)
- Training infra, metrics (wandb, lightning, ...)

Training infrastructure

Domain libraries

PyTorch / JAX

Kernels and frameworks

NCCL / MPI / system libraries

CUDA toolkit

Drivers and hardware

Zoom #1: Python packaging

- sdist vs. wheels
- Version constraints
- Building from source is often very brittle
- No native packaging solution for native libraries
- Python packages make assumption about what is available on the system

Zoom #2: Library loading on Linux

When a program (Python script, executable, library) needs to *access* a dependency on Linux, there are three ways:

- Static linking

Zoom #2: Library loading on Linux

When a program (Python script, executable, library) needs to *access* a dependency on Linux, there are three ways:

- Static linking
- Dynamic linking

Zoom #2: Library loading on Linux

When a program (Python script, executable, library) needs to *access* a dependency on Linux, there are three ways:

- Static linking
- Dynamic linking
- YOLO **dlopening**

ImportError: libcudnn.so.7: cannot open shared object file: No such file or directory

Can Nix help?

Important properties:

- Explicit thanks to hermeticity

Important properties:

- Explicit thanks to hermeticity
- Reproducible
- Everything^{*} is captured in the DAG
 - (almost) no assumptions on the host system

Important properties:

- Explicit thanks to hermeticity
- Reproducible
- Everything^{*} is captured in the DAG
 - (almost) no assumptions on the host system
- Everything is source-built

Important properties:

- Explicit thanks to hermeticity
- Reproducible
- Everything* is captured in the DAG
 - (almost) no assumptions on the host system
- Everything is source-built
- Caching makes it efficient



How Nix handles shared libraries

In general, *surgery* is often needed.

How Nix handles shared libraries

In general, *surgery* is often needed.

- Static linking: `buildInputs`

How Nix handles shared libraries

In general, *surgery* is often needed.

- Static linking: `buildInputs`
- Dynamic linking: `patchelf` hooks + custom loader and static RPATH
 - No uncertainty

How Nix handles shared libraries

In general, *surgery* is often needed.

- Static linking: `buildInputs`
- Dynamic linking: `patchelf` hooks + custom loader and static RPATH
 - No uncertainty
- YOLO `dlopen`s. Those need to be patched, but once they are, they become robust.

How Nix handles shared libraries

In general, *surgery* is often needed.

- Static linking: `buildInputs`
- Dynamic linking: `patchelf` hooks + custom loader and static RPATH
 - No uncertainty
- YOLO `dlopen`s. Those need to be patched, but once they are, they become robust.

Bonus: NVIDIA driver (`libcuda.so` & co):

- Mounted at `/run/opengl-driver`
- On non-NixOS systems: NixGL

Alternatives?

Isn't uv the go-to solution now?

- uv is great! (really)
- However, it is pure-python -> Falls short when native code / source-compilation comes into play

What about containers?

- Not meant for the same problem
- Good for distribution
- Not reproducible

Nix can build containers too!

Drawbacks of Nix:

- Packaging is hard is required
 - LLMs can help to some extent
- Evaluation is still slow (painful on large projects)

Future improvements:

- Dynamic derivations for finer grained builds
- Better solutions for the enterprise (suited for large scale clusters, etc.)
- castore approaches for more efficient caching and serving

More generally, Nix's paradigm suits very well the *agentic* workflow.

We're hiring!



Join me at Mistral!

We work on cool and hard problems, at scale
(research, engineering, sysadmins, etc.)

<https://mistral.ai/careers/>

